Cybersecurity

RESEARCH

Open Access

CL-SCCA: multi-byte contrastive learning for side-channel collision attacks on cryptosystems

Zoushaojie Jiang[1,2], An Wang[1,3], Annyu Liu[1*], Yaoling Ding[1*], Jingqi Zhang[1], Jiakun Li[1] and Liehuang Zhu[1]

Abstract

Side-channel collision attacks (SCCA) are a classical method for recovering key differences by detecting collisions between intermediate values. Traditional SCCA methods usually rely on manually designed statistical distinguishers to measure the similarity between traces of different bytes, making it difficult to fully exploit the hidden shared leakage features among different byte traces under noise and nonlinear leakage. Deep learning-based SCCA improves the ability to extract leakage features, but existing methods are usually trained around a single byte and therefore cannot fully utilize the richer leakage distribution provided by multi-byte traces. To address this issue, this paper proposed a Contrastive Learning-based SCCA method, called CL-SCCA. CL-SCCA uses traces corresponding to multiple byte operations for pretraining, enabling the encoder to capture similar leakage features from the multi-byte leakage distribution, thereby improving key-difference recovery. Furthermore, this paper proposed a byte-weighted contrastive loss function to reduce the influence of byte-source differences on model training during pretraining. Across datasets covering different cryptographic primitives and hardware platforms, the proposed method consistently achieved better attack performance, with relative improvements in recovery success rate of approximately 14.88%, 77.73%, and 65.20% on ASCAD_F, ASCAD_R, and DPA Contest v4, respectively.

Keywords Side-channel analysis, Deep learning, Side-channel collision attack, Contrastive learning

Introduction

Side-channel analysis (SCA) infers sensitive information processed inside cryptographic devices by collecting information generated during their operation, such as power consumption, electromagnetic radiation, and timing delay (Kocher 1996; Kocher et al. 1999; Agrawal et al. 2002). According to the prior information available to the adversary, SCA can generally be divided into two categories: profiled SCA and non-profiled SCA. Profiled SCA typically assumes that the adversary has access to a fully controllable profiling device that is identical or similar to the target device. A leakage model is then constructed in advance and transferred to the traces collected from the target device for key recovery. Representative methods include template attacks and deep-learning-based profiled SCA (Choudary and Kuhn 2017; Cao et al. 2022). In contrast, non-profiled SCA does not rely on an additional profiling device, making it more realistic in many practical attack scenarios (Timon 2019).

Side-channel collision attacks (SCCA), first introduced by Schramm et al. (2003), are a representative type of non-profiled SCA. The basic idea of SCCA is to detect whether intermediate-value collisions occur between different internal operations, and then recover

*Correspondence:
Annyu Liu
annvliu@bit.edu.cn
Yaoling Ding
dyl19@bit.edu.cn
[1] School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China
[2] State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao 266237, China
[3] Shandong Key Laboratory of Energy Industry Internet Big Data Technology, Jinan 250003, Shandong, China

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

the relationships among key bytes without directly guessing the specific value of the target key (Bogdanov 2007). However, traditional SCCA usually relies on manually designed statistical distinguishers to measure the similarity or correlation between traces of different bytes, so as to determine whether they leak the same intermediate value. When facing complex noise and nonlinear leakage, these methods struggle to exploit the hidden shared leakage features among traces of different bytes, resulting in unstable collision-relation recovery. Therefore, improving the ability of SCCA to model complex leakage features in non-profiled scenarios remains an important problem in this research direction.

In recent years, deep learning has attracted extensive attention in SCA due to its excellent capability for nonlinear feature extraction (Ou and Li 2022; Zaid et al. 2021; Benadjila et al. 2020). Inspired by this, (Staib and Moradi 2023) proposed the Deep Learning-based Side-channel Collision Attack (DL-SCCA). This method exploits the one-to-one mapping between plaintexts and intermediate values under a fixed-key setting. It trains a neural network using the plaintext of a selected byte as the label, so that the model extracts intermediate-value-related leakage features from the traces corresponding to this byte. The trained model is then transferred to the traces of other bytes to recover key differences.

DL-SCCA improves the ability of SCCA to model complex leakage features, but its training is usually centered on a single byte. The reason is that the subkeys corresponding to different bytes are usually different, and the same plaintext label may correspond to different intermediate values in different bytes. If traces from multiple bytes are directly used for joint training according to plaintext labels, the model will have difficulty determining which leakage features should be learned under the same label. Therefore, DL-SCCA can hardly make full use of the richer leakage distribution and similar leakage features provided by multi-byte traces, which limits its ability to recover cross-byte key differences.

Contrastive learning (CL) provides a solution to the above problem. CL does not rely on labels. Instead, it pulls together different augmented samples from the same trace and separates samples from different traces, enabling the model to learn more discriminative and robust leakage features during unsupervised training (Chen et al. 2020; Liu et al. 2025). Therefore, this paper proposed a CL-based SCCA method, called CL-SCCA. The proposed method performs contrastive training using unlabeled traces corresponding to multiple byte operations, allowing the model to capture similar leakage features from the multi-byte leakage distribution. In this way, it enhances the utilization of multi-byte leakage information and improves key-difference recovery.

The main contributions of this paper are summarized as follows:

- We proposed a CL-based SCCA method, called CL-SCCA. The proposed method uses traces corresponding to multiple byte operations, enabling the model to capture similar leakage features from the multi-byte leakage distribution. This enhances the model's ability to utilize multi-byte leakage information and improves key-difference recovery.
- We proposed combining CL with a simple neural network structure, enabling the model to effectively extract collision-related leakage features without relying on a complex network architecture, thereby improving the performance of SCCA.
- We proposed a byte-weighted contrastive loss function, which assigns different weights in the contrastive loss according to the byte source of each trace. This reduces the influence of byte-source differences on model training and improves cross-byte key-difference recovery.
- We validated the effectiveness of CL-SCCA on three public datasets, including ASCAD_F, ASCAD_R, and DPA Contest v4. Compared with DL-SCCA, CL-SCCA achieves relative improvements in recovery success rate of approximately 14.88%, 77.73%, and 65.20%, respectively, and obtains better key-difference ranking results.

The remainder of this paper is organized as follows. Section "Related work" reviews the related work. Section "Preliminaries" provides background knowledge on SCCA, DL-SCCA, and CL. Section "Contrastive learning-based side-channel collision attack" details the proposed method. Section "Experiments" presents the experimental results. Section "Discussion" discusses the influence of leakage window size, the applicability of CL-SCCA to different cryptographic primitives, and its robustness against common side-channel countermeasures. Section "Conclusion" concludes this paper.

Related work

Our work is related to a series of studies on SCCA. Schramm et al. (2003) first applied the idea of SCCA to DES. Subsequently, (Bogdanov 2007) extended this idea to AES by identifying collisions through the consistency between related leakages in different executions, which significantly reduced the number of traces required for the attack. Moradi et al. (2010) proposed a further correlation-enhanced collision analysis method, which uses correlation coefficients as the collision distinguisher and extends collision analysis to masked AES implementations with first-order leakage. These studies detect

collisions by analyzing the correlation between leakages from different operations, and recover key relations accordingly. However, due to temporal offsets and distortions in leakage features, directly using correlation for collision detection may be problematic.

Our work is also related to a series of studies that use deep learning techniques to improve SCA performance. Deep learning was first widely applied to profiled SCA. In this attack setting, the adversary typically has access to a controllable profiling device that is identical or similar to the target device and uses labeled profiling traces to train a leakage model. Maghrebi et al. (2016) first introduced neural networks into profiled SCA to automatically extract leakage features from traces. Cagli et al. (2017) combined CNNs with data augmentation techniques to improve the attack capability of models under misalignment and perturbations. Benadjila et al. (2020) released the ASCAD dataset, providing a public evaluation basis for deep-learning-based SCA. Based on these studies, (Jin et al. 2020) further explored network architectures suitable for SCA and introduced the convolutional block attention module into CNNs, enabling the model to focus on more relevant leakage positions in input traces. These studies show that deep learning can effectively improve the extraction of leakage features in SCA, providing a new methodological basis for handling more complex side-channel leakages. Liu et al. (2026) further proposed a collision-based profiled SCA method, which transforms key recovery into a collision detection problem. This method uses neural networks to determine whether intermediate-value collisions occur between different traces, and recovers key differences based on collision probabilities. These studies show that deep learning can effectively enhance the extraction of collision-related leakage features, providing a methodological basis for further improving SCCA performance.

Non-profiled SCA does not rely on an additional profiling device and conducts the analysis using only traces collected from the target device together with known input and output information. SCCA is a representative type of non-profiled SCA. Inspired by the above studies, researchers have further attempted to use deep learning to improve the ability of SCCA to model complex leakage features. Staib and Moradi (2023) introduced deep learning into SCCA, using neural networks to learn the leakage features of different intermediate data processing operations from trace segments. The trained neural network is then applied to another trace segment, and key differences are recovered by comparing the leakage features of different trace segments. Wu et al. (2024) directly built a plaintext-based model from raw traces to characterize the leakage of intermediate values, and revealed all key differences by comparing the leakage produced by the model without directly processing the traces.

In recent years, self-supervised learning techniques, including contrastive learning, have gradually been introduced into SCA to enhance the model's ability to extract leakage features. As shown in Table 1, (Kong et al. 2023) used a self-supervised autoencoder to reconstruct raw traces, thereby suppressing noise and enhancing leakage features. Liu et al. (2025) proposed a CL-based profiled SCA method that extracts similarities and dissimilarities among traces in an unsupervised setting and subsequently uses labeled traces to achieve robust classification, with the aim of suppressing irrelevant information while preserving key classification features. In addition to these self-supervised methods, (Jiang et al. 2025) proposed a profiled SCA method based on supervised contrastive learning. This method uses labels of sensitive intermediate values to guide the construction of similarity relationships among samples, enabling the model to learn more discriminative leakage features.

More recently, (Llavata et al. 2026) applied Barlow Twins-based self-supervised learning to horizontal collision analysis for asymmetric cryptography. Their samples consist of repeated modular-operation segments extracted from the same trace, and the objective is to determine whether collisions occur between the operands corresponding to different segments. In contrast, the samples considered in this paper are obtained from different byte operations across multiple executions of a symmetric cipher. The objective is to transfer collision-related leakage learned from one byte

Table 1 Comparison of CL-SCCA with existing works

Work	Attack scenario	Learning mechanism	Cipher type	Collision analysis
Kong et al. (2023)	Non-profiled SCA	Autoencoder	Symmetric	No
Liu et al. (2025)	Profiled SCA	Unsupervised CL	Symmetric	No
Jiang et al. (2025)	Profiled SCA	Supervised CL	Symmetric	No
Llavata et al. (2026)	Non-profiled SCA	Barlow Twins	Asymmetric	Yes
CL-SCCA	Non-profiled SCCA	Unsupervised CL	Symmetric	Yes

to other bytes and recover inter-byte key differences using known input relationships. Because the two categories of methods differ in terms of cipher type, sample construction, collision relationships, and leakage characteristics, this work must additionally address inter-byte differences in temporal position, amplitude, morphology, and masking state to achieve stable cross-byte transfer. Therefore, how to extract transferable similar leakage features from unlabeled multi-byte traces remains an open problem.

Preliminaries

Side-channel collision attack

SCCA is a representative non-profiled SCA method. Its core idea is to recover key relations by detecting collisions between leakages corresponding to different operations during encryption.

Taking the AES SubBytes operation as an example, as shown in Fig. 1, let p_i denote the i-th byte of the plaintext and k_i denote the i-th byte of the key. If the leakage features in the traces corresponding to two S-box outputs are similar, they can be regarded as processing the same intermediate variable, which can be expressed as

$$\begin{aligned} S(p_i \oplus k_i) = S\left(p_j \oplus k_j\right) \Rightarrow p_i \oplus k_i = p_j \\ \oplus k_j \Rightarrow k_i \oplus k_j = p_i \oplus p_j. \end{aligned} \tag{1}$$

Accordingly, the difference between k_i and k_j can be inferred as $\Delta k_{ij} = k_i \oplus k_j$. Unlike SCA methods that directly guess a single key byte, SCCA first recovers the difference relations among different key bytes. Once these differences are determined, only one reference subkey byte needs to be further enumerated, and the remaining bytes can be derived accordingly. Therefore, the key space to be searched can be significantly reduced to 2^8.

In practical attacks, collisions cannot be directly observed. Therefore, it is usually necessary to construct a collision score for each candidate difference δ, and rank the candidates according to the correlation between traces of different bytes or other statistical distinguishers, which is given by

$$\widehat{\Delta k}_{ij} = \arg\max_{\delta \in \mathbb{F}_2^8} \text{Score}(\delta). \tag{2}$$

If $\delta = \Delta k_{ij}$, the corresponding sample pairs are more likely to satisfy the collision condition and thus obtain a higher collision score.

Deep learning-based side-channel collision attack

DL-SCCA (Staib and Moradi 2023) introduces deep learning into SCCA. It learns the mapping between the traces of a fixed byte and the corresponding plaintext labels, and transfers the learned leakage features to the traces of other bytes. The key difference is then recovered according to the relationship between the model output and the known plaintext.

Specifically, let t_i denote the trace corresponding to the S-box output of the i-th byte, p_i denote the corresponding plaintext byte, and k_i denote the corresponding key byte. Under a fixed-key setting, there is a one-to-one correspondence between p_i and the intermediate value $S(p_i \oplus k_i)$. Therefore, the neural network $M(\cdot)$ can be trained using (t_i, p_i), so that it learns the mapping between the training traces and the plaintext labels, which is expressed as $M(t_i) \rightarrow p_i$.

After training, the model is applied to the trace t_j corresponding to the S-box output of another byte j, producing the model output $\hat{y} = M(t_j)$. Since the model is trained on the i-th byte, the output $\hat{y}$ can be interpreted as the equivalent plaintext value of t_j under the model of the i-th byte. In other words, the leakage features of t_j are the closest to those produced by the i-th byte when the plaintext is $\hat{y}$. If this matching relationship holds, the corresponding S-box inputs satisfy

$$\hat{y} \oplus k_i = p_j \oplus k_j. \tag{3}$$

Thus, the difference relation between the two key bytes can be obtained as $\Delta k_{ij} = k_i \oplus k_j = \hat{y} \oplus p_j$. Therefore, DL-SCCA essentially transforms collision detection into a neural-output-based key-difference recovery problem by learning leakage features from traces.

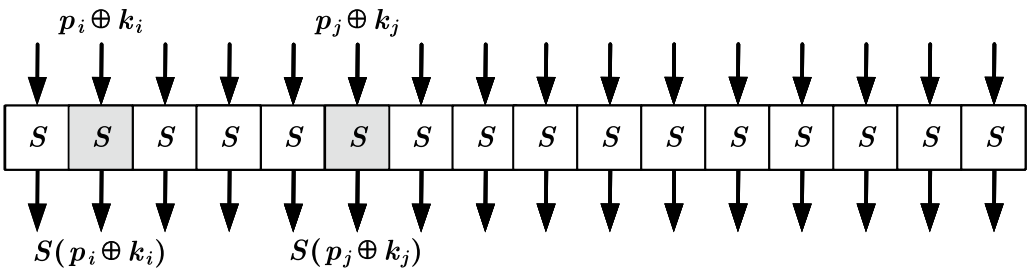


Fig. 1 Illustration of collisions in the AES SubBytes operation

Contrastive learning

CL is a feature representation learning method based on comparing the similarities among samples. Its core idea is to pull positive sample pairs closer in the representation space while pushing negative sample pairs farther apart, thereby obtaining more stable and discriminative feature representations (Oord et al. 2018; Chen et al. 2020; Wu et al. 2018). In this way, CL can exploit the structural information inherent in the data for representation learning without relying on manually annotated labels.

As shown in Fig. 2, let x_i and x_j denote two augmented samples generated from the same original trace through two random transformations. These two samples form a positive pair. Meanwhile, an augmented sample x_k generated from another original trace is taken as a negative sample. The encoder $\varphi(\cdot)$ maps these samples into the representation space, producing the corresponding representations $\varphi(x_i)$, $\varphi(x_j)$, and $\varphi(x_k)$. The objective of CL is to make the positive pair as close as possible in the representation space while separating it from the negative sample, i.e.,

$$d\big(\varphi(x_i), \varphi(x_j)\big) < d(\varphi(x_i), \varphi(x_k)), \tag{4}$$

where $d(\cdot, \cdot)$ denotes the distance metric in the representation space.

Contrastive learning-based side-channel collision attack

Before applying CL-SCCA, this paper follows DL-SCCA (Staib and Moradi 2023) and uses the sensitivity analysis in (Timon 2019) to extract the leakage intervals corresponding to each byte. Based on these intervals, the full traces are segmented to obtain trace sets associated with different bytes. The overall procedure of the proposed CL-SCCA method is shown in Fig. 3, and mainly includes three stages, namely pretraining, fine-tuning, and key recovery. Specifically, CL-SCCA first uses unlabeled traces corresponding to multiple bytes for pretraining, enabling the model to capture similar leakage features from the multi-byte leakage distribution. Subsequently, following the training and key recovery procedure of DL-SCCA, this paper fine-tunes the model using the plaintext labels of a selected byte, and transfers the fine-tuned model to the traces of other bytes to guess key differences. The specific details of the proposed method are further described in the following subsections.

Procedure of CL-SCCA

After trace segmentation, CL-SCCA first uses unlabeled traces corresponding to multiple bytes for pretraining, aiming to enhance the model's ability to extract similar leakage features from the multi-byte leakage distribution. Specifically, taking AES as an example, let the trace sets obtained after segmentation be $\{T_s\}_{s=0}^{15}$, where T_s denotes the trace set corresponding to the s-th byte. This paper selects trace sets corresponding to multiple bytes to construct the pretraining dataset D_p, which contains N_p unlabeled traces. The pretraining process is performed in a mini-batch manner. In each mini-batch, data augmentation, feature encoding, projection mapping, and contrastive optimization are sequentially conducted as follows.

(1) *Data augmentation.* For each pretraining trace t_i in a mini-batch, two random data augmentations are applied to generate two augmented samples x_i and x_j, which are treated as a positive pair. Meanwhile, augmented samples from other pretraining traces in the same batch are used as negative samples during training. This paper does not specifically design new data augmentation strategies, but adopts the random data augmentation operations proposed in Liu et al. (2025), including Random Shift (RS), Random Cropping (RC), and Random Denoising (RD), to enhance the robustness of the model against noise and local perturbations.

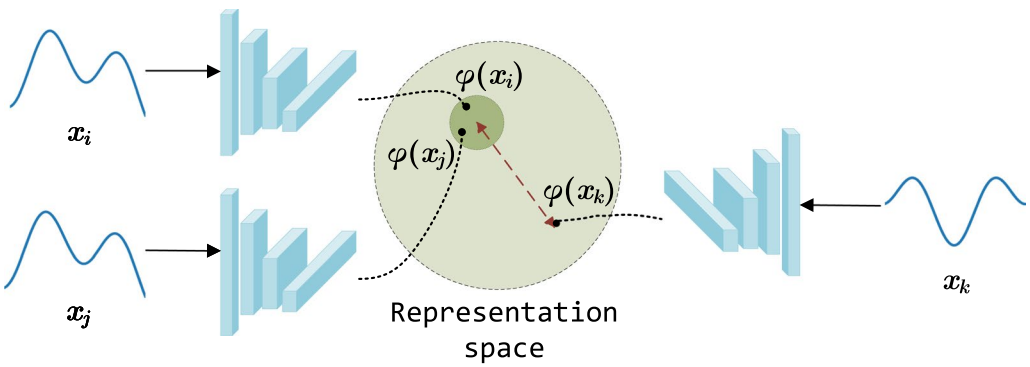


Fig. 2 The process of contrastive learning

(2) *Feature encoding.* The augmented samples are fed into the encoder $\varphi(\cdot)$, producing the corresponding feature vectors $h_i = \varphi(x_i)$ and $h_j = \varphi(x_j)$. This paper uses a simple multilayer perceptron (MLP) as the encoder. The MLP consists of four fully connected layers, each followed by a ReLU activation function, and the hidden dimension is set to 1024. This paper combines CL with a simple neural network structure rather than relying on a more complex network architecture, enabling the model to effectively extract collision-related leakage features with relatively low model complexity.

(3) *Projection mapping.* A projection head $g(\cdot)$ is added after the encoder to further map the features, producing the projection vectors $z_i = g(h_i)$ and $z_j = g(h_j)$ in the contrastive space. The projection head consists of two fully connected layers and one ReLU activation function, with the hidden dimension set to 128, so as to enhance the nonlinear expressiveness of the features. The projection head is used only during pretraining and is removed in the subsequent fine-tuning and key recovery stage.

(4) *Contrastive optimization.* Inspired by Liu et al. (2025), this paper adopts the normalized temperature-scaled cross-entropy loss (NT-Xent) to optimize the projection vectors z_i and z_j. This loss improves the similarity between different augmented samples from the same trace and distinguishes samples from different traces, allowing the model to learn more stable and discriminative leakage features. For any two projection vectors z_i and z_k, the similarity function is defined as

$$s_{ik} = \mathrm{sim}(z_i, z_k) = \frac{z_i^T z_k}{\|z_i\|_2 \|z_k\|_2}. \tag{5}$$

For a mini-batch containing n original traces, $2n$ projection vectors are obtained after two data augmentations. Let $z_{j(i)}$ denote the positive sample corresponding to z_i. The overall contrastive loss is defined as

$$\mathcal{L}_{\text{NT-Xent}} = -\frac{1}{2n}\sum_{i=1}^{2n} \log \frac{\exp\left(\mathrm{sim}\left(z_i, z_{j(i)}\right)/\tau\right)}{\sum_{k=1}^{2n} \mathbb{I}_{[k\neq i]} \exp\left(\mathrm{sim}(z_i, z_k)/\tau\right)}, \tag{6}$$

where τ denotes the temperature coefficient and is set to 0.07. $\mathbb{I}_{[k\neq i]}$ is the indicator function, with $\mathbb{I}_{[k\neq i]} = 1$ when $k \neq i$, and $\mathbb{I}_{[k\neq i]} = 0$ when $k = i$. Through the above pretraining, the encoder $\varphi(\cdot)$ can capture similar leakage features from traces corresponding to multiple bytes without relying on plaintext labels, thereby providing a better model initialization for subsequent key-difference recovery.

After pretraining, this paper removes the projection head $g(\cdot)$ and adds a fully connected layer after the pretrained encoder $\varphi(\cdot)$ as the classification head $f(\cdot)$, whose output dimension is set to 256. The model is then fine-tuned using the trace set T_r corresponding to the training byte r and its plaintext labels p_r. The cross-entropy loss is used to optimize the model parameters, enabling the model to adapt to the downstream collision analysis task based on plaintext labels. In this stage, the model outputs a 256-dimensional probability vector.

After fine-tuning, this paper follows the key recovery procedure of DL-SCCA (Staib and Moradi 2023) and transfers the fine-tuned model to the trace set T_q corresponding to the target byte q. For the m-th target trace $t_q^{(m)}$, the model outputs the probability vector

$$\mathbf{P}_q^{(m)} = \left\{P_q^{(m)}(v)\right\}_{v=0}^{255}, \tag{7}$$

where $P_q^{(m)}(v)$ denotes the probability that the model output class is v. When the candidate key difference is δ, the model output should satisfy $v = p_q^{(m)} \oplus \delta$. Therefore, this paper accumulates the log-probabilities over all attack traces as

$$Y_{rq}(\delta) = \sum_{m=1}^{N_a} \log P_q^{(m)}\left(\delta \oplus p_q^{(m)}\right), \tag{8}$$

where N_a denotes the number of traces used in the key recovery stage. Finally, all candidate differences are ranked, and the result corresponding to the maximum statistic is selected as the estimated key difference

$$\widehat{\Delta k}_{rq} = \arg\max_{\delta \in \mathbb{F}_2^8} Y_{rq}(\delta). \tag{9}$$

Byte-weighted CL-SCCA

In multi-byte pretraining, traces corresponding to different bytes may exhibit positional, amplitude, and morphological differences. Moreover, when a mini-batch contains samples from multiple bytes in approximately equal numbers, cross-byte negative samples usually outnumber same-byte negative samples. To analyze the roles of these two types of negative samples in contrastive optimization, this paper divides the negative samples associated with the anchor z_i into a same-byte negative sample set and a cross-byte negative sample set:

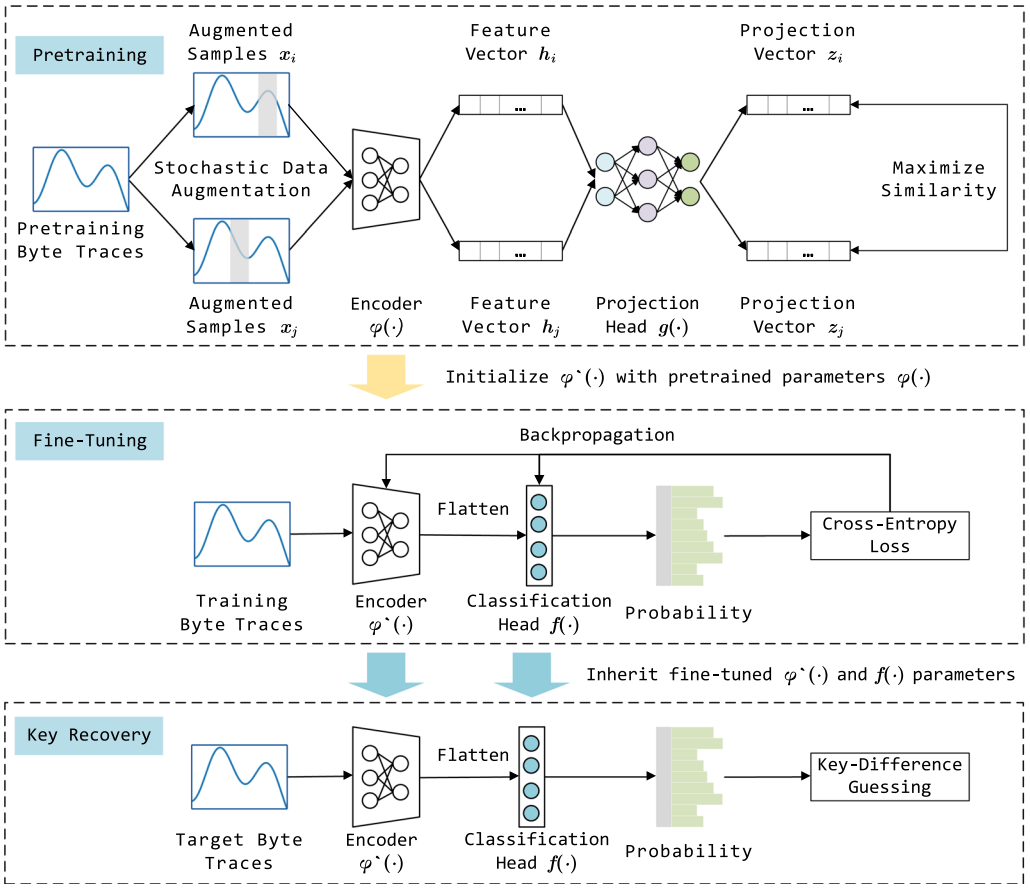


Fig. 3 Overall procedure of the CL-SCCA method

$$\begin{aligned} \mathcal{N}_i^{\text{same}} &= \{k \mid k \neq i,\ k \neq j(i),\ b_k = b_i\}, \\ \mathcal{N}_i^{\text{cross}} &= \{k \mid k \neq i,\ k \neq j(i),\ b_k \neq b_i\}, \end{aligned} \tag{10}$$

where b_i denotes the byte index associated with the projection vector z_i, and $z_{j(i)}$ denotes its positive sample. Using the similarity defined in Eq. (5), the single-sample loss corresponding to the anchor z_i in the original NT-Xent loss can be written as

$$\ell_i^{\text{NT-Xent}} = -\log \frac{\exp\left(s_{i,j(i)}/\tau\right)}{\exp\left(s_{i,j(i)}/\tau\right) + \sum_{k\in\mathcal{N}_i^{\text{same}}} \exp\left(s_{ik}/\tau\right) + \sum_{k\in\mathcal{N}_i^{\text{cross}}} \exp\left(s_{ik}/\tau\right)}. \tag{11}$$

The original NT-Xent loss assigns equal weights to the two types of negative samples. For same-byte negative samples, because they originate from the same byte, the model cannot distinguish them solely on the basis of byte position, amplitude, or overall trace morphology and must further extract fine-grained leakage differences. In contrast, cross-byte negative samples may exhibit more pronounced byte-source differences, allowing the model to separate them by exploiting these differences. When cross-byte negative samples are numerically dominant and participate in optimization with the same weight as same-byte negative samples, their cumulative contribution may bias the model toward learning byte-source information rather than sufficiently extracting collision-related leakage features shared across bytes. Therefore, this paper defines the negative-sample weights according to the byte sources of the negative sample and the anchor:

$$w_{ik} = \begin{cases} 1, & b_i = b_k, \\ \alpha, & b_i \neq b_k, \end{cases} \tag{12}$$

where α is the cross-byte weighting coefficient satisfying $0 < \alpha \leq 1$. Based on these weights, the weighted denominator associated with the anchor z_i is defined as

$$D_i = \exp\left(s_{i,j(i)}/\tau\right) + \sum_{k \in \mathcal{N}_i^{\text{same}}} \exp\left(s_{ik}/\tau\right) + \alpha \sum_{k \in \mathcal{N}_i^{\text{cross}}} \exp\left(s_{ik}/\tau\right). \tag{13}$$

Based on D_i, the single-sample loss associated with the anchor z_i is defined as

$$\ell_i^{\text{BWCL}} = -\log \frac{\exp\left(s_{i,j(i)}/\tau\right)}{D_i}. \tag{14}$$

Accordingly, the Byte-Weighted Contrastive Loss (BWCL) is defined as

$$\mathcal{L}_{\text{BWCL}} = \frac{1}{2n} \sum_{i=1}^{2n} \ell_i^{\text{BWCL}} = -\frac{1}{2n} \sum_{i=1}^{2n} \log \frac{\exp\left(s_{i,j(i)}/\tau\right)}{D_i}, \tag{15}$$

where τ denotes the temperature coefficient, and $2n$ denotes the total number of augmented samples in a mini-batch. For any negative sample z_k, the gradient of the single-sample loss with respect to the similarity s_{ik} is

$$\frac{\partial \ell_i^{\text{BWCL}}}{\partial s_{ik}} = \frac{w_{ik} \exp\left(s_{ik}/\tau\right)}{\tau D_i}. \tag{16}$$

To further analyze the effect of BWCL on network parameter updates, let the last fully connected layer of the projection head be $z_a = Wr_a + \boldsymbol{\beta}$, where r_a denotes the feature vector of sample a input to this layer, and W and $\boldsymbol{\beta}$ denote the weight matrix and bias of this layer, respectively. Let the gradients of the cosine similarity with respect to the two projection vectors be $c_{ik}^{(i)} = \partial s_{ik}/\partial z_i$ and $c_{ik}^{(k)} = \partial s_{ik}/\partial z_k$. According to the chain rule, the contribution of the negative pair (i, k) to the gradient with respect to W is

$$\left.\frac{\partial \ell_i^{\text{BWCL}}}{\partial W}\right|_{(i,k)} = \frac{w_{ik} \exp\left(s_{ik}/\tau\right)}{\tau D_i} \left[c_{ik}^{(i)} r_i^T + c_{ik}^{(k)} r_k^T\right]. \tag{17}$$

For same-byte negative samples, $w_{ik} = 1$, whereas for cross-byte negative samples, $w_{ik} = \alpha$. Therefore, BWCL uses α to scale the pairwise backpropagation coefficient of cross-byte negative samples, thereby reducing their contribution to parameter updates.

Although BWCL reduces the weights assigned to cross-byte negative samples, samples from other bytes remain involved in pretraining. Traces from each byte can serve as anchors and form positive pairs with their augmented views, allowing the shared encoder to learn their leakage features. Meanwhile, cross-byte negative samples still participate in contrastive optimization with the weight α. Therefore, BWCL does not force features from different bytes to become closer. Instead, it reduces the tendency of the model to excessively separate cross-byte samples, encouraging it to focus more on similar leakage features that can be shared across traces from different bytes.

Based on the above loss design, this paper replaces the original NT-Xent loss with BWCL during the pretraining stage of CL-SCCA, while the pretraining, fine-tuning, and key-recovery procedures remain unchanged. The resulting method is termed Byte-Weighted CL-SCCA, abbreviated as BWCL-SCCA. When $\alpha = 1$, BWCL-SCCA reduces to CL-SCCA using the original NT-Xent loss.

Experiments

All experiments in this paper are conducted in a Python 3.9 environment and implemented using PyTorch. The experiments are run on a server equipped with 256 GB memory and two NVIDIA A100-SXM4 GPUs with 80 GB memory each. We use the key-rank, defined as the rank index of the correct key difference Δk_{ij} in $Y_{ij}(\delta)$, as the metric for evaluating the success of deep-learning-based SCA. A smaller key-rank indicates that the correct key difference is ranked closer to the top. When the key-rank decreases to 0, the correct key difference ranks first among all candidates, indicating a successful attack.

To obtain fair results, this paper randomly shuffles the order of attack traces 100 times during the attack stage, and computes the key-rank curve with respect to the number of attack traces for each shuffled order. The average of the 100 key-rank curves is then used as the final key-rank curve of the model. Furthermore, $N_{\text{KR}=0}$ denotes the number of attack traces required for the average key-rank curve to reach 0 for the first time. If the curve does not reach 0 within the full range of available attack traces, the maximum number of available attack traces is assigned to $N_{\text{KR}=0}$ for statistical analysis. For each experimental setting, three models are trained independently, and the mean and standard deviation of the three results are reported. This paper does not specifically tune the network hyperparameters, but adopts

Table 2 Hyperparameter settings

Pretraining	
Optimizer	Adam
Learning rate	1×10^{-5}
Learning rate scheduler	CosineAnnealingLR
Loss functions	{NT-Xent, BWCL}
Batch size	200
Epochs	200
Fine-tuning	
Optimizer	Adam
Learning rate	1×10^{-5}
Learning rate scheduler	OneCycleLR
Loss function	Cross-entropy loss
Batch size	200
Epochs	200

a unified hyperparameter setting in all experiments, as shown in Table 2. Except for the ablation experiments on the weighting coefficient, the weighting coefficient α in BWCL is set to 0.05.

Datasets

This paper evaluates the effectiveness of CL-SCCA on five side-channel datasets, including three public AES datasets, namely ASCAD_F, ASCAD_R, and DPA Contest v4, as well as the SM4 and PRESENT datasets collected in this work. These datasets cover three block ciphers, including AES, SM4, and PRESENT, and different hardware platforms, including 8-bit microcontrollers, 16-bit smart cards, and 32-bit microcontrollers. They are used to evaluate the applicability of CL-SCCA across different cryptographic primitives, S-box structures, and hardware implementations. The details of these datasets are described below.

(1) *ASCAD_F.* ASCAD_F (ANSSI 2026) is the fixed-key version of the ASCAD database. It was collected from a Boolean-masked AES implementation running on an 8-bit ATMega8515 microcontroller (Benadjila et al. 2020). This dataset contains 60,000 synchronized electromagnetic traces, and each trace consists of 100,000 sampling points covering the first round of AES encryption.

(2) *ASCAD_R.* ASCAD_R (ASCAD 2026) is the random-key version of the ASCAD database, which was also collected from a masked AES implementation on the ATMega8515 (Benadjila et al. 2020). Unlike ASCAD_F, this dataset contains 300,000 electromagnetic traces. Among every three acquisitions, one uses a fixed key, while the other two use random keys. Each trace contains 250,000 sampling points and covers the first two rounds of AES encryption. Therefore, the 100,000 traces collected with the fixed key can be used in our method.

(3) *DPA Contest v4.* DPA Contest v4 (Dpa Contest 2026) was collected from an Atmel ATMega-163 smart card running a masked AES-256 implementation. This implementation adopts rotating S-box masking to improve the efficiency of masking while maintaining security. The dataset contains 100,000 power traces in total, and each trace consists of 435,000 sampling points covering the first round of AES and the beginning of the second round.

(4) *SM4.* SM4 is a block cipher with a 128-bit block size and a 128-bit key. Its round function includes byte-level nonlinear S-box operations. In this work, SM4 is implemented on a 16-bit smart card, and 10,000 power traces are collected at a sampling frequency of 25 MHz, with each trace containing 15,500 sampling points. The analysis mainly focuses on the power leakage associated with the S-box operations in the first round of SM4, based on which cross-byte key-difference recovery experiments are conducted.

(5) *PRESENT-128.* PRESENT is a lightweight block cipher designed for resource-constrained devices. It adopts a 64-bit block size and a substitution–permutation structure based on a 4-bit S-box. This work uses PRESENT-128, namely a PRESENT implementation with a 128-bit key, and collects 20,000 power traces from an STM32F-429BIT6 microcontroller. The sampling frequency is 125 MHz, and each trace contains 24,000 sampling points. The attack experiments mainly target the S-box operations in the first round of PRESENT-128.

Results on ASCAD_F

This section evaluates the attack performance of the proposed method on the ASCAD_F dataset. It should be noted that the first two bytes in the ASCAD_F and ASCAD_R datasets are not masked and exhibit relatively simple leakage. Therefore, they are excluded from the attack analysis in this paper. In the pretraining stage, CL-SCCA uses the traces of bytes 2, 3, 4, and 10 for training, with 20,000 traces selected for each byte. In the fine-tuning stage, 20,000 traces from byte 10 are selected to train the model, which is then transferred to other target bytes to recover key differences.

To validate the effectiveness of CL-SCCA, this section compares four methods, including DL-SCCA (Staib and Moradi 2023), MLP without CL, CL-SCCA with MLP,

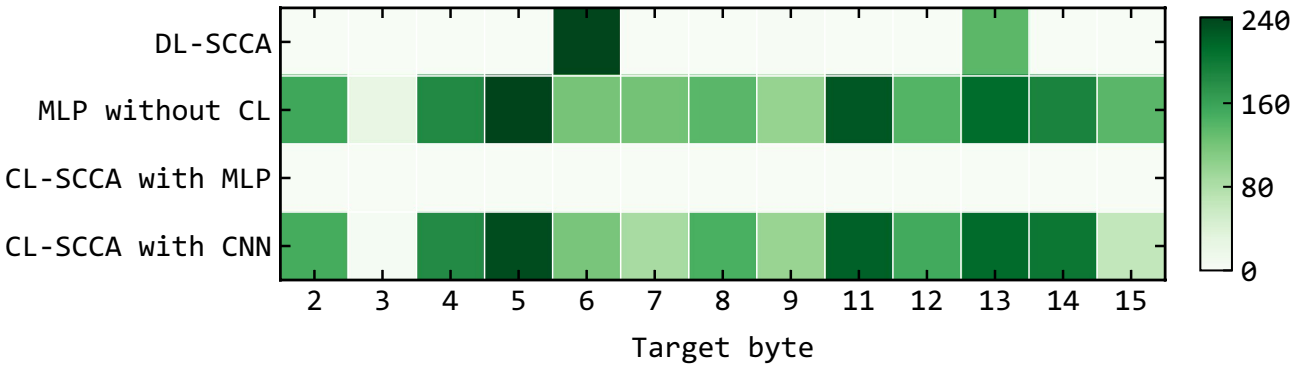
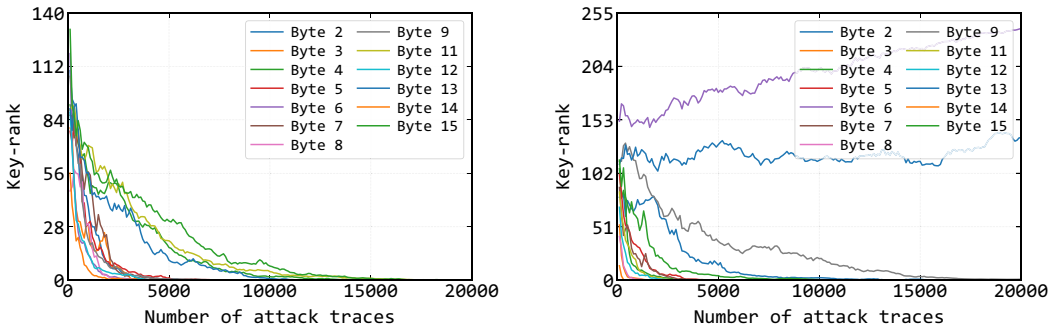


Fig. 4 Key-rank heatmaps of different methods on ASCAD_F



(a) CL-SCCA with MLP (b) DL-SCCA

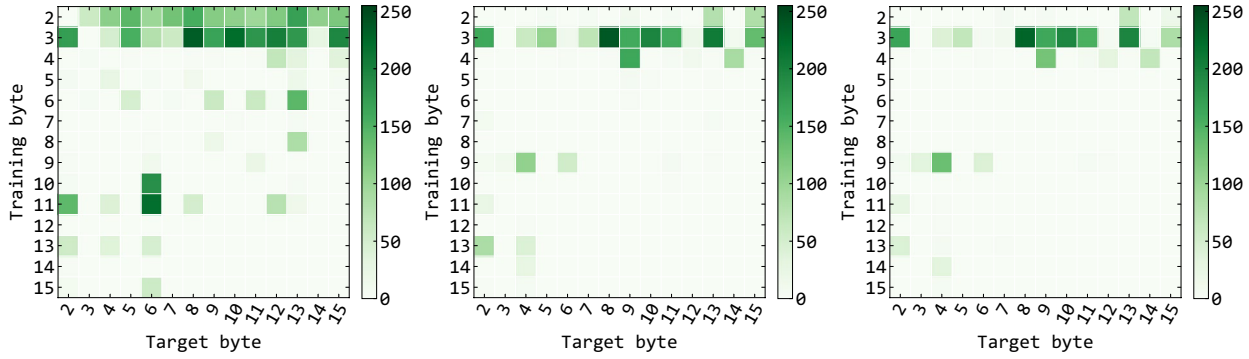
Fig. 5 Key-rank evolution with the number of attack traces on ASCAD_F

and CL-SCCA with CNN. The MLP without CL and CL-SCCA with MLP use the same MLP architecture, and the only difference lies in whether CL is introduced. CL-SCCA with CNN uses the same network architecture as DL-SCCA as the encoder.

Figure 4 shows the key-rank heatmaps of the four methods. It can be observed that CL-SCCA with MLP achieves a key-rank of 0 for all target bytes and successfully recovers all key differences, thereby reducing the remaining key space to 2^8. In contrast, the MLP without CL still has high key-rank values for most target bytes, indicating that directly performing supervised training with a simple MLP is insufficient for effectively extracting cross-byte collision-related leakage features. DL-SCCA can recover the correct key differences for some target bytes, but still produces high ranks for several bytes. The performance of CL-SCCA with CNN is not better than that of CL-SCCA with MLP, suggesting that the improvement on ASCAD_F mainly comes from the enhanced leakage feature extraction brought by CL, rather than from a more complex network architecture.

Figure 5 further shows the key-rank curves of DL-SCCA and CL-SCCA with MLP as the number of attack traces increases. For CL-SCCA with MLP, the key-rank values of all target bytes quickly converge to 0 within 20,000 attack traces, and the convergence process is relatively stable. In contrast, the key-rank values of some target bytes in DL-SCCA decrease more slowly, and even remain high as the number of attack traces increases. These results show that CL-SCCA not only improves the final key-difference ranking, but also reduces the number of attack traces required for key-difference recovery, thereby improving the attack efficiency.

Based on this observation, Fig. 6 presents the key-difference recovery results under different combinations of training bytes and target bytes. To ensure a fair comparison, BWCL-SCCA uses the same MLP architecture as CL-SCCA with MLP, and this implementation is denoted as BWCL-SCCA with MLP. The horizontal axis represents the target byte, the vertical axis represents the training byte, and the color represents the key-rank. Overall, CL-SCCA with MLP can reduce the key-rank to 0 for most byte pairs and clearly outperforms DL-SCCA. This indicates that multi-byte pretraining improves the model's ability to utilize similar leakage features across bytes. BWCL-SCCA with MLP further reduces some



(a) DL-SCCA (b) CL-SCCA with MLP (c) BWCL-SCCA with MLP

Fig. 6 Average key-rank heatmaps over three independent runs under multi-byte training and attack on ASCAD_F

high-key-rank regions compared with CL-SCCA with MLP, suggesting that the byte-weighted contrastive loss function can reduce the influence of byte-source differences on model training to some extent, thereby improving the success rate of cross-byte key-difference recovery. Finally, Table 3 summarizes the means and standard deviations of the average key-rank and recovery success rate obtained by the three methods over three independent runs on ASCAD_F, further validating the advantages of CL-SCCA in overall key-difference recovery performance and experimental stability.

Results on ASCAD_R

This section further evaluates the attack performance of the proposed method on the ASCAD_R dataset. Similar to ASCAD_F, in the pretraining stage, CL-SCCA uses the traces of bytes 2 to 5 for training, with 40,000 traces selected for each byte. In the fine-tuning stage, 40,000 traces from byte 8 are selected to train the model, which is then transferred to other target bytes to recover key differences. To validate the effectiveness of CL-SCCA, this section compares the results of four methods on ASCAD_R, including DL-SCCA, MLP without CL, CL-SCCA with MLP, and CL-SCCA with CNN.

Figure 7 shows the key-rank heatmaps of the four methods. It can be observed that CL-SCCA with MLP reduces the key-rank to 0 for all target bytes and clearly outperforms the other three methods. In contrast, the MLP without CL still has high key-rank values for most target bytes, indicating that training with a simple MLP alone is insufficient for stably extracting cross-byte collision-related leakage features. DL-SCCA successfully

Table 3 Performance comparison of different methods on ASCAD_F over three independent runs

Metric	DL-SCCA (Staib and Moradi 2023)		CL-SCCA with MLP		BWCL-SCCA with MLP	
	Mean	Std	Mean	Std	Mean	Std
Average key-rank	28.18	4.86	13.15	1.02	**10.92**	0.61
Success rate (%)	70.15	2.47	73.81	2.47	**80.59**	1.30

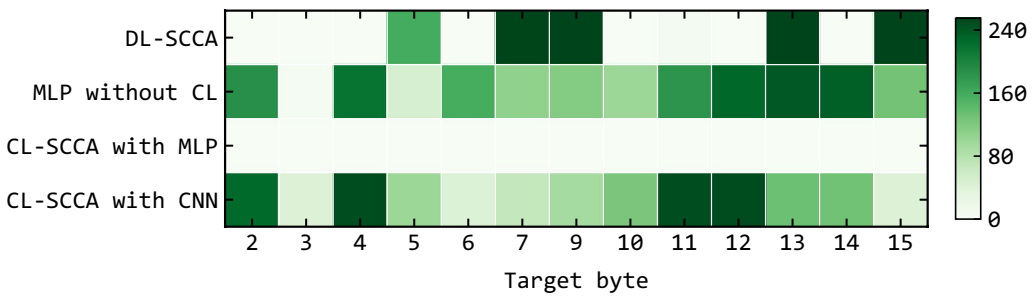
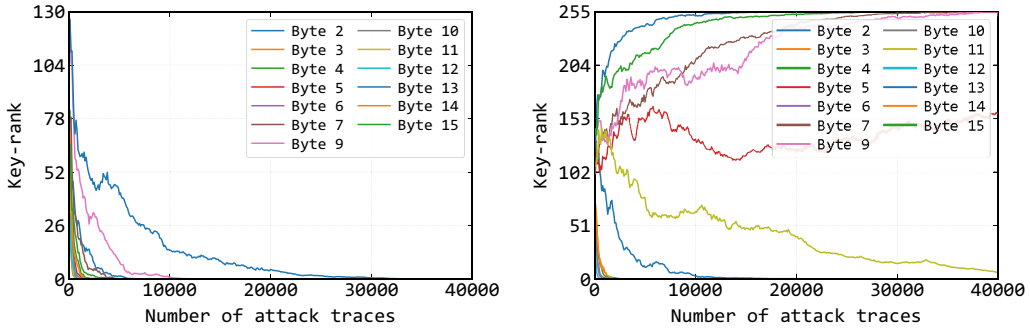


Fig. 7 Key-rank heatmaps of different methods on ASCAD_R



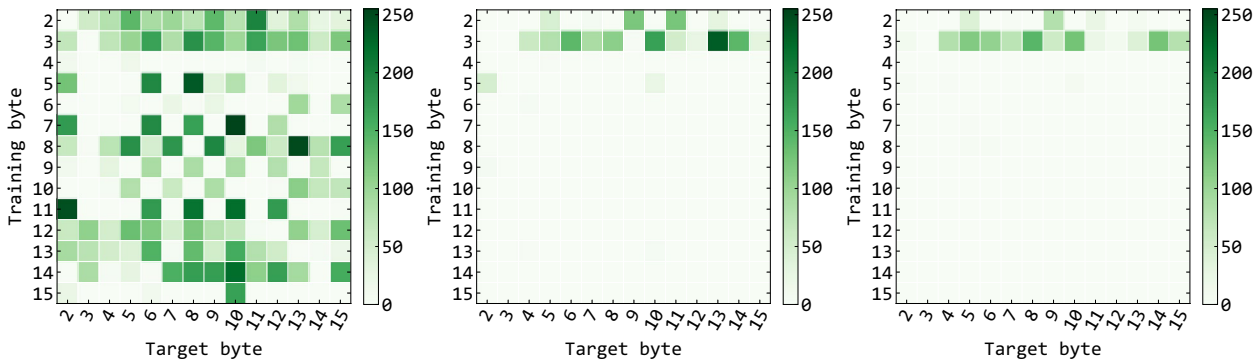
(a) CL-SCCA with MLP (b) DL-SCCA

Fig. 8 Key-rank evolution with the number of attack traces on ASCAD_R

recovers the correct key differences for some target bytes, but still produces high ranks for multiple target bytes. The results of CL-SCCA with CNN are also not better than those of CL-SCCA with MLP, further indicating that the performance improvement of CL-SCCA mainly comes from the enhanced leakage feature extraction brought by CL.

Figure 8 further shows the key-rank curves of DL-SCCA and CL-SCCA with MLP as the number of attack traces increases. The key-rank values of CL-SCCA with MLP decrease faster overall, and all target bytes stably converge to 0 as the number of attack traces increases. In contrast, the key-rank values of some target bytes in DL-SCCA still fail to converge to 0 within 40,000 attack traces. This result shows that CL-SCCA not only improves the final key-rank, but also reduces the number of attack traces required for key-difference recovery.

Figure 9 presents the key-difference recovery results under different combinations of training bytes and target bytes. Overall, DL-SCCA still has clear high-key-rank regions for many byte pairs, indicating its limited success rate in cross-byte recovery. In contrast, CL-SCCA with MLP significantly reduces the key-rank for most byte pairs, validating the effectiveness of multi-byte pretraining in improving the utilization of similar leakage features across bytes. On the ASCAD_R dataset, BWCL-SCCA with MLP provides only a slight improvement in overall success rate compared with CL-SCCA, because CL-SCCA with MLP has already achieved a high recovery success rate and leaves limited room for further improvement. Finally, Table 4 summarizes the means and standard deviations of the average key-rank and recovery success rate obtained by the three methods over three independent runs on ASCAD_R. Compared with DL-SCCA, BWCL-SCCA with MLP increases the mean recovery success rate by 39.29 percentage points, corresponding to a relative improvement of approximately 77.73%, while also reducing the average key-rank. These results further validate the effectiveness of CL-SCCA on ASCAD_R.



(a) DL-SCCA (b) CL-SCCA with MLP (c) BWCL-SCCA with MLP

Fig. 9 Average key-rank heatmaps over three independent runs under multi-byte training and attack on ASCAD_R

Table 4 Performance comparison of different methods on ASCAD_R over three independent runs

Metric	DL-SCCA (Staib and Moradi 2023)		CL-SCCA with MLP		BWCL-SCCA with MLP	
	Mean	Std	Mean	Std	Mean	Std
Average key-rank	63.80	14.24	8.53	0.19	**6.33**	0.38
Success rate (%)	50.55	7.05	88.10	0.69	**89.84**	0.67

Results on DPA Contest v4

This section further evaluates the attack performance of the proposed method on the DPA Contest v4 dataset. In the pretraining stage, CL-SCCA uses the traces of bytes 0 and 1 for training, with 5000 traces selected for each byte. In the fine-tuning stage, 5000 traces from byte 0 are selected to train the model, which is then transferred to other target bytes to recover key differences.

Figure 10 shows the key-rank heatmaps of the four methods. It can be observed that CL-SCCA with MLP significantly reduces the key-rank for most target bytes compared with DL-SCCA and the MLP without CL, indicating that multi-byte contrastive pretraining can still improve the model's ability to extract similar leakage features. However, unlike the ASCAD_F and ASCAD_R datasets, CL-SCCA with MLP still produces relatively high key-rank values for some target bytes, suggesting that cross-byte transfer is more difficult on DPA Contest v4. The results of CL-SCCA with CNN are not better than those of CL-SCCA with MLP, which further indicates that the performance improvement on this dataset still mainly comes from the enhanced leakage feature extraction brought by CL, rather than from a more complex network architecture.

Figure 11 further shows the key-rank curves of DL-SCCA and CL-SCCA with MLP as the number of attack traces increases. Compared with DL-SCCA, CL-SCCA with MLP can reduce the key-rank more rapidly for some target bytes and achieve recovery with fewer attack traces. However, the key-rank values of some target bytes still do not converge to 0 as the number of attack traces increases. This may be because sensitivity analysis does not accurately locate the effective leakage intervals corresponding to these bytes, which indicates that the leakage features of these bytes and the training byte are difficult for the model to align stably.

Figure 12 presents the key-difference recovery results under different combinations of training bytes and target bytes. It can be observed that DL-SCCA still produces high key-rank values for a large number of byte

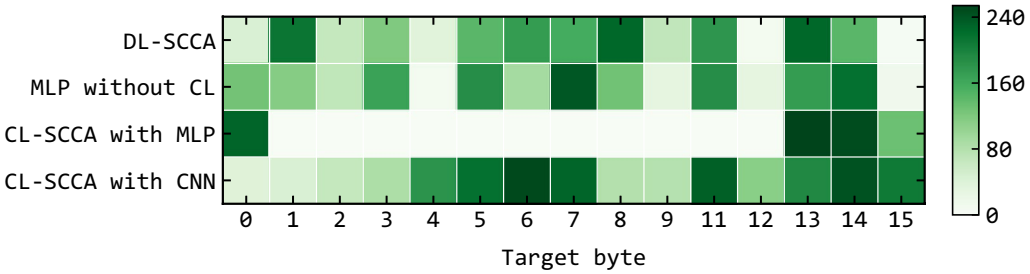
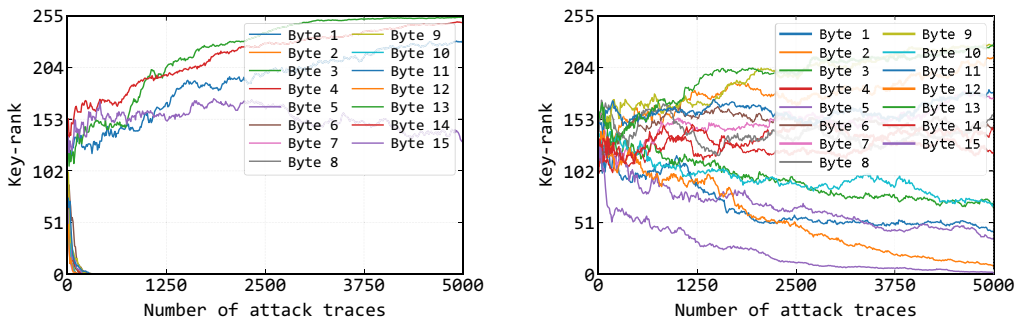
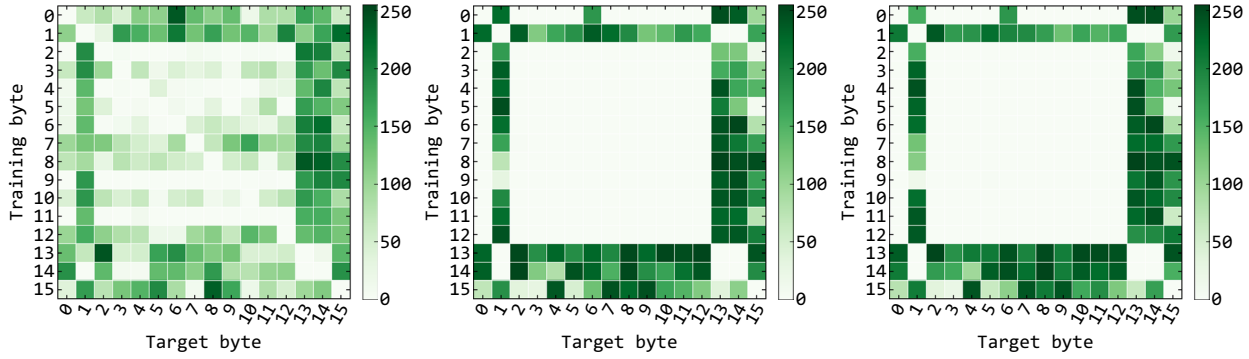


Fig. 10 Key-rank heatmaps of different methods on DPA contest v4



(a) CL-SCCA with MLP (b) DL-SCCA

Fig. 11 Key-rank evolution with the number of attack traces on DPA contest v4



(a) DL-SCCA (b) CL-SCCA with MLP (c) BWCL-SCCA with MLP

Fig. 12 Average key-rank heatmaps over three runs for multi-byte training and attack on DPA contest v4

pairs, indicating its relatively limited key-difference recovery capability on DPA Contest v4. In contrast, CL-SCCA with MLP substantially enlarges the low-key-rank regions and successfully recovers the key differences for more byte pairs, demonstrating that multi-byte pretraining improves the model's ability to extract transferable leakage features.

However, as shown in Table 5, the overall recovery results of BWCL-SCCA with MLP are close to those of CL-SCCA with MLP, with only a slight improvement in $N_{\mathrm{KR}=0}$. Further examination of Fig. 12 shows that the recovery results on DPA Contest v4 are not uniformly distributed across all byte pairs, but exhibit a clear grouping structure. Key differences between bytes within the same group are relatively easy to recover, whereas the key-rank values between different byte groups are generally high. Some bytes also exhibit limited transferability to other bytes.

To analyze this phenomenon, the byte-specific leakage traces extracted based on the sensitivity analysis are grouped according to the values of the masked S-box output and the output-mask phase, respectively, and the average trace of each group is calculated at each sampling point. The group-averaged traces corresponding to different variable values for the same byte are then concatenated in sequence and standardized. Pearson correlation coefficients are subsequently calculated to measure the similarity of leakage morphology between different bytes. As shown in Fig. 13, the inter-byte correlations of the group-averaged traces obtained using the masked S-box output and the output-mask phase exhibit grouping structures consistent with the key-difference recovery results. The group-averaged traces of bytes within the same group have relatively high correlations, whereas the correlations between different byte groups are substantially lower. The group-averaged traces of some bytes are nearly uncorrelated with those of the other bytes. These results indicate that cross-byte leakage in DPA Contest v4 is not uniformly shared, but exhibits both within-group similarity and between-group differences. Byte pairs with large differences in their group-averaged traces are generally difficult to transfer effectively, which is consistent with their high key-rank values.

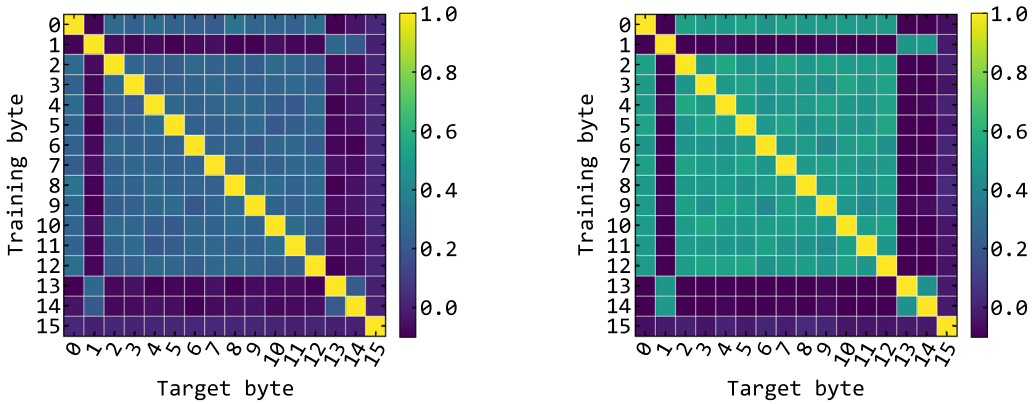
The above analysis further explains why BWCL-SCCA with MLP and CL-SCCA with MLP achieve similar overall performance. BWCL can preserve transferable leakage features by reducing the repulsive effect of cross-byte negative samples, but it cannot eliminate the inherent leakage differences between different byte groups. Therefore, the two methods achieve the same recovery success rate, while BWCL mainly provides a slight improvement in $N_{\mathrm{KR}=0}$.

Comparison of self-supervised learning-based side-channel analysis methods

To compare CL-SCCA with other self-supervised learning-based SCA methods, this section selects two representative feature-learning techniques, namely a Siamese network and an autoencoder (Kong et al. 2023), as comparison baselines and conducts

Table 5 Performance comparison of different methods on DPA contest v4 over three independent runs

Metric	DL-SCCA (Staib and Moradi 2023)		CL-SCCA with MLP		BWCL-SCCA with MLP	
	Mean	Std	Mean	Std	Mean	Std
Average key-rank	80.70	7.81	**74.47**	2.35	75.58	3.47
$N_{\mathrm{KR}=0}$	3368.25	268.60	2386.43	10.78	**2380.13**	23.27
Success rate (%)	36.03	2.94	**59.52**	0.00	**59.52**	0.00



(a) Masked S-box output (b) Output-mask phase

Fig. 13 Byte-wise Pearson correlations of group-averaged traces under different leakage variables on DPA contest v4

experiments on ASCAD_F. Specifically, the Siamese network and autoencoder are used to replace the contrastive pretraining module of CL-SCCA, while the subsequent classification fine-tuning and key-difference recovery procedures remain unchanged. The Siamese network adopts the same network architecture as CL-SCCA as its shared encoder, and the pretrained encoder parameters are transferred to the classification fine-tuning stage. The autoencoder adopts the encoder architecture in Kong et al. (2023), processes the original traces through a reconstruction task, and feeds the reconstructed traces into the classifier for training and attack. Except for the method-specific training mechanisms, the same experimental settings are used to reduce the influence of other factors on the comparison results.

As shown in Table 6, the autoencoder-based method produces a high average key-rank and a recovery success rate below 1%, indicating that trace reconstruction cannot directly obtain discriminative features suitable for cross-byte key-difference recovery. The Siamese network outperforms the autoencoder-based method, but its average key-rank remains high, and its recovery success rate is only approximately 15%. In contrast, BWCL-SCCA with MLP achieves the lowest average key-rank and $N_{\mathrm{KR}=0}$, together with a recovery success rate exceeding 80%, demonstrating better attack performance across all three metrics.

The autoencoder is optimized to reconstruct the original traces and therefore tends to focus on the overall trace structure and dominant energy distribution. During reconstruction, fine-grained leakage features related to collision discrimination may be smoothed or attenuated, and the reconstructed traces may therefore fail to preserve sufficient discriminative information. The performance of the Siamese network is mainly constrained by its sample-pair construction strategy. This method uses plaintext labels to determine positive and negative sample pairs. However, for different bytes, even identical plaintext values usually correspond to different subkeys and therefore different S-box intermediate values. If such traces are still treated as positive pairs, the model is forced to pull together leakage features associated with different intermediate values, thereby introducing incorrect similarity constraints. Consequently, the Siamese network cannot directly exploit traces from multiple bytes for joint training, which limits its utilization of multi-byte leakage information. In addition, this method constructs sample pairs directly from plaintext labels without introducing data augmentation to improve its adaptability to noise, misalignment, and local perturbations, thereby limiting the generalizability of the learned features. In contrast,

Table 6 Comparison of self-supervised learning-based SCA methods on ASCAD_F over three independent runs

Method	$N_{\mathrm{KR}=0}$		Average key-rank		Success rate (%)	
	Mean	Std	Mean	Std	Mean	Std
Autoencoder-based (Kong et al. 2023)	19992.67	3.17	128.77	6.06	0.55	0.00
Siamese network	19477.11	231.82	66.36	8.19	14.79	5.50
BWCL-SCCA with MLP	**12,760.07**	312.96	**10.92**	0.61	**80.59**	1.30

CL-SCCA performs pretraining using data augmentation and traces from multiple bytes, enabling the model to learn features that are more robust to noise, misalignment, and local perturbations from richer leakage distributions, thereby achieving better generalization and cross-byte transferability.

Hyperparameter evaluation

This section further investigates the influence of different hyperparameters on the attack performance of CL-SCCA on ASCAD_F, including the data augmentation strategy, the number of pretraining bytes, and the cross-byte weighting coefficient α. The experiment on the number of pretraining bytes is further used to verify the contribution of multi-byte pretraining to cross-byte leakage feature learning and key-difference recovery. Except for the factor being evaluated, all other hyperparameters are kept unchanged, and three models are independently trained for each setting.

Effect of data augmentation strategies

To evaluate the influence of different data augmentation operations on the attack performance of CL-SCCA, this paper tests RC, RD, and RS individually and in different combinations. As shown in Table 7, without data augmentation, the model exhibits a high average key-rank and a low recovery success rate, indicating that data augmentation plays an important role in learning leakage features transferable across bytes. Using either RS or RC alone substantially improves the attack performance, whereas using RD alone does not provide an effective improvement, indicating clear differences in the contributions of the three augmentation operations.

Among the combined settings, jointly using RS and RC achieves strong overall performance. Its average key-rank is nearly identical to that obtained using all three augmentations, while its recovery success rate is higher, indicating that RS and RC provide complementary benefits in improving feature robustness. To maintain consistent experimental configurations across different datasets, the main experiments in this paper still adopt the complete combination of RC, RD, and RS.

Table 7 Average key-rank and success rate under different data augmentation combinations on ASCAD_F

Number of types	Types	Average key-rank		Success rate (%)	
		Mean	Std	Mean	Std
None	–	80.20	10.02	8.79	5.06
One type	RC	23.18	2.21	54.40	3.14
	RD	87.34	8.52	5.49	1.79
	RS	20.12	4.73	61.90	8.42
Two types	RD + RS	20.19	2.04	64.84	1.35
	RD + RC	22.05	2.58	54.58	1.81
	RS + RC	13.16	0.73	**76.74**	0.69
Three types	RD + RS + RC	**13.15**	1.02	73.81	2.47

Effect of the number of pretraining bytes

To evaluate the influence of the number of pretraining bytes on attack performance and further verify the contribution of multi-byte pretraining to cross-byte leakage feature learning and key-difference recovery, this paper varies the number of bytes used during pretraining, denoted by B. Under each setting, 20,000 traces are provided for each byte, resulting in a total of $20,000B$ pretraining traces, with $B = 1$ serving as the single-byte pretraining baseline. As shown in Table 8, as the number of pretraining bytes increases, the average key-rank continuously decreases and the recovery success rate generally increases. The setting $B = 8$ achieves the best attack performance, indicating that increasing the number of pretraining bytes exposes the encoder to richer leakage distributions and improves subsequent cross-byte key-difference recovery.

However, increasing the number of pretraining bytes also enlarges the pretraining dataset, increases storage and memory requirements, and raises both the computational cost per epoch and the overall pretraining time. Although $B = 8$ achieves the best attack results, $B = 4$ already provides a substantial performance improvement over single-byte and two-byte pretraining while requiring lower storage and computational costs. Therefore, to balance key-difference recovery performance and pretraining cost, this paper ultimately selects four bytes for pretraining CL-SCCA on ASCAD_F.

Effect of the weighting coefficient α

This paper further investigates the influence of the cross-byte weighting coefficient α on attack performance by conducting experiments with different values of α on ASCAD_F. A smaller α reduces the contribution of cross-byte negative samples to contrastive optimization. When $\alpha = 1$, BWCL reduces to the original NT-Xent loss. Suppose that a mini-batch contains B bytes and M

Table 8 Average key-rank and success rate under different numbers of pretraining bytes on ASCAD_F

B	Pretraining bytes	Average key-rank		Success rate (%)	
		Mean	Std	Mean	Std
1	{2}	31.19	2.01	39.38	2.60
2	{2, 10}	16.16	1.31	70.70	2.60
4	{2, 3, 4, 10}	13.15	1.02	73.81	2.47
8	{3, 4, 5, 6, 7, 8, 9, 10}	**8.15**	1.07	**86.63**	1.27

augmented samples for each byte. The numbers of same-byte and cross-byte negative samples corresponding to an arbitrary anchor are $M-2$ and $(B-1)M$, respectively. To approximately balance the cumulative contributions of the two types of negative samples, the recommended initial value of α can be defined as

$$\alpha_{\text{rec}} = \frac{|\mathcal{N}_i^{\text{same}}|}{|\mathcal{N}_i^{\text{cross}}|} = \frac{M-2}{(B-1)M} \approx \frac{1}{B-1}. \tag{18}$$

When four bytes are used for pretraining, this value is approximately 1/3.

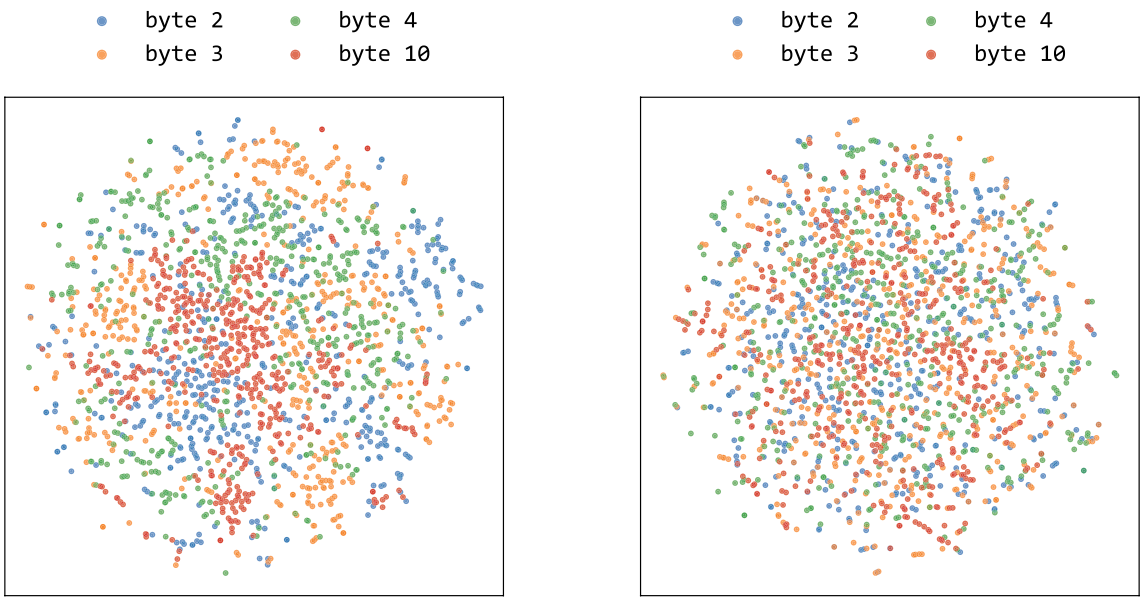
Table 9 Average key-rank and success rate under different weighting coefficients α on ASCAD_F

α	Average key-rank		Success rate (%)	
	Mean	Std	Mean	Std
0.01	10.79	0.32	79.67	1.19
0.05	10.92	0.61	80.59	1.30
0.10	**10.54**	0.51	80.59	2.26
0.20	12.10	1.16	78.02	2.06
0.25	11.71	0.40	78.39	1.81
0.33	11.94	0.49	**80.95**	1.04
0.50	13.11	0.83	74.73	1.19
1.00	13.15	1.02	73.81	2.47

As shown in Table 9, compared with $\alpha = 1$, reducing the weight of cross-byte negative samples generally decreases the average key-rank and increases the recovery success rate, indicating that weakening the repulsive effect of cross-byte negative samples benefits cross-byte key-difference recovery. When $\alpha \leq 0.33$, the experimental results obtained under different settings are relatively close, demonstrating that BWCL is reasonably robust to the choice of α within this range. Considering both the average key-rank and recovery success rate, the performance obtained with $\alpha = 0.05$ is close to that of the best-performing setting. Therefore, $\alpha = 0.05$ is adopted in the subsequent experiments.

To further analyze the influence of α on representation quality, this paper performs t-SNE visualization of the projection vectors z obtained with $\alpha = 1$ and $\alpha = 0.05$. Specifically, 500 test traces are randomly selected from each of bytes 2, 3, 4, and 10, and the resulting 2,000 projection vectors are mapped into a two-dimensional space using the same parameters and random seed. The t-SNE algorithm uses PCA initialization with the perplexity set to 30.

As shown in Fig. 14, when $\alpha = 1$, projection vectors from different bytes exhibit clear byte-specific clustering in some regions, indicating that the representations obtained through equal-weight optimization retain substantial byte-source information. When $\alpha = 0.05$, the overlap among samples from different bytes increases, while the independent clustering of samples from the same byte becomes less pronounced. This indicates



(a) $\alpha = 1.00$ (b) $\alpha = 0.05$

Fig. 14 t-SNE visualization of the projection vectors obtained with different cross-byte weighting coefficients on ASCAD_F

that reducing the weight of cross-byte negative samples weakens the influence of byte-source differences on the representation space and encourages the model to focus more on leakage features shared across different bytes. Together with the attack results reported in Table 9, the adjustment of the representation space introduced by BWCL further improves the transferability of the learned features for cross-byte key-difference recovery.

Discussion

Influence of leakage window size

This section further investigates the influence of leakage-window size on the attack performance of CL-SCCA by conducting experiments with leakage windows of different lengths on ASCAD_F. Except for the window size, all other experimental configurations are kept unchanged, and three models are independently trained for each setting. As shown in Table 10, the leakage-window size has a clear impact on attack performance, which does not improve monotonically as the window increases. With 700 sampling points, the model achieves the highest recovery success rate and exhibits relatively small experimental variation. When the window is extended to 1000 sampling points, the average key-rank increases substantially and the recovery success rate decreases markedly, indicating that the additional trace region may contain considerable information unrelated to the target leakage, thereby increasing the difficulty of feature extraction.

When the window is further extended to 1500 and 2000 sampling points, the average key-rank decreases to some extent, but the recovery success rates remain lower than that obtained with 700 sampling points. This suggests that a larger window may improve candidate ranking for some byte pairs, whereas the additionally introduced irrelevant leakage and noise increase the difficulty of cross-byte feature extraction, making it difficult for the correct key difference to consistently reach a key-rank of 0. Therefore, this paper adopts a leakage window of 700 sampling points.

Table 10 Average key-rank and success rate under different leakage window sizes on ASCAD_F

Window size	Average key-rank		Success rate (%)	
	Mean	Std	Mean	Std
700	13.15	1.02	73.81	2.47
1000	25.40	1.90	49.27	7.52
1500	9.68	1.29	35.35	5.24
2000	8.02	0.61	51.47	5.24

Results on SM4

To verify the applicability of CL-SCCA to different cryptographic primitives, this paper further conducts key-difference recovery experiments on the SM4 dataset and compares three methods: DL-SCCA, CL-SCCA with MLP, and BWCL-SCCA with MLP. Table 11 reports the average key-rank, $N_{\mathrm{KR}=0}$, and recovery success rate of the three methods.

The experimental results show that CL-SCCA with MLP and BWCL-SCCA with MLP achieve comparable average key-rank values and recovery success rates, both substantially outperforming DL-SCCA. This demonstrates that multi-byte contrastive pretraining can effectively extract transferable collision-related leakage features across different bytes of SM4. Compared with CL-SCCA with MLP, BWCL-SCCA with MLP achieves a slightly lower $N_{\mathrm{KR}=0}$, indicating that the byte-weighted contrastive loss can reduce, to some extent, the number of attack traces required for successful recovery while maintaining the key-difference ranking and recovery success rate.

The SM4 experiments extend the evaluation of CL-SCCA to a non-AES cryptographic primitive and a different hardware implementation. The results demonstrate that its multi-byte contrastive learning mechanism is applicable to cross-byte key-difference recovery under different cipher structures and provide further experimental evidence for the generalizability of CL-SCCA.

Results on PRESENT-128

This paper further conducts key-difference recovery experiments on the PRESENT-128 dataset to evaluate the applicability of CL-SCCA to lightweight ciphers with 4-bit S-boxes. Since PRESENT-128 employs a 4-bit S-box, the classification output is set to 16 classes, and the candidate key-difference space consists of 16

Table 11 Performance comparison of different methods on SM4

Method	$N_{\mathrm{KR}=0}$		Average key-rank		Success rate (%)	
	Mean	Std	Mean	Std	Mean	Std
DL-SCCA	7141.94	155.36	15.64	10.04	50.00	11.79
CL-SCCA with MLP	5009.17	665.57	0.19	0.08	88.89	3.93
BWCL-SCCA with MLP	4940.28	521.31	0.33	0.27	88.89	7.86

possible values. Table 12 summarizes the average key-rank, $N_{KR=0}$, and recovery success rate of the different methods.

The experimental results show that CL-SCCA with MLP and BWCL-SCCA with MLP achieve comparable attack performance and both generally outperform DL-SCCA. The two methods more effectively improve the ranking of the correct key difference and enhance cross-byte key-difference recovery, indicating that multi-byte feature learning can still extract transferable collision-related leakage features in the 4-bit S-box setting. Together with the SM4 experiments, the results on PRESENT-128 further demonstrate that the applicability of CL-SCCA is not limited to cipher implementations using 8-bit S-boxes, but can be extended to symmetric ciphers with different cipher structures, S-box granularities, and hardware platforms, providing further experimental evidence for its applicability to different cryptographic primitives.

Robustness against common side-channel countermeasures

The aforementioned ASCAD_F, ASCAD_R, and DPA Contest v4 datasets all employ masking countermeasures, and the corresponding experiments have verified the attack performance of CL-SCCA in masked scenarios. This section further evaluates the robustness of CL-SCCA against two common side-channel countermeasures, namely desynchronization and random delay. Desynchronization is introduced by applying an independent random offset to the leakage window of each byte in ASCAD_F, causing the target leakage to be misaligned within the window. Random delay is introduced by inserting zero-valued segments of random lengths into the byte-specific traces in ASCAD_F, thereby changing the relative timing among local sampling points. All three methods use the network architecture shown in Table 13 and are denoted as CL-SCCA with CNN_{TD}, DL-SCCA with CNN_{TD}, and BWCL-SCCA with CNN_{TD}, respectively.

Table 14 summarizes the average key-rank and recovery success rate of the three methods under the two countermeasures. Under desynchronization, CL-SCCA with CNN_{TD} achieves better overall attack performance than DL-SCCA with CNN_{TD}, indicating that multi-byte contrastive pretraining improves the adaptability of the model to changes in leakage positions. Random delay further changes the relative distances among local peaks, resulting in nonuniform temporal deformation of the traces. In this scenario, BWCL-SCCA with CNN_{TD} achieves a slightly lower average key-rank, whereas CL-SCCA with CNN_{TD} still achieves the highest recovery success rate, and both methods outperform DL-SCCA with CNN_{TD}.

BWCL-SCCA with CNN_{TD} does not exhibit a consistent advantage across the two countermeasures, possibly because desynchronization and random delay cause the differences among samples to be determined more strongly by changes in leakage positions and local trace morphology. In this case, traces from the same byte may also exhibit substantial differences, while traces from different bytes may still retain similar local structures. Therefore, fixed weights based on byte indices may not accurately reflect the actual relationships among perturbed samples and may also weaken some effective cross-byte constraints.

Overall, CL-SCCA with CNN_{TD} achieves the highest recovery success rate under both desynchronization and random delay, while its average key-rank is also lower than that of DL-SCCA with CNN_{TD} in both scenarios. These results demonstrate that the proposed CL-SCCA improves the adaptability of SCCA to temporal perturbations and maintains better overall key-difference recovery performance under the two common side-channel countermeasures.

Table 13 Architecture of CNN_{TD} used in the robustness experiments

CNN_{TD} Architecture	
Convolutional blocks	4
Filters	{32, 64, 128, 128}
Kernel Sizes	{21, 15, 11, 7}
Block Structure	Conv1D + BatchNorm + ReLU
Pooling Layers	3
Pooling	Average, kernel size 2
Feature Aggregation	Adaptive average and max pooling
Fully-connected Layers	2
Neurons	{512, 128}
Dropout	0.25
Output Dimension	256

Table 12 Performance comparison of different methods on PRESENT-128

Method	**$N_{KR=0}$**		**Average key-rank**		**Success rate (%)**	
	Mean	**Std**	**Mean**	**Std**	**Mean**	**Std**
DL-SCCA	7369.94	637.61	3.44	0.33	66.67	3.25
CL-SCCA with MLP	5798.36	518.31	1.60	0.41	75.83	4.26
BWCL-SCCA with MLP	5841.49	434.65	1.70	0.31	75.56	3.49

Table 14 Performance comparison under desynchronization and random delay

Countermeasure	Method	Average key-rank	Success rate (%)
Desynchronization	DL-SCCA with CNN_{TD}	111.23	3.85
	CL-SCCA with CNN_{TD}	9.04	70.88
	BWCL-SCCA with CNN_{TD}	13.16	60.99
Random delay	DL-SCCA with CNN_{TD}	128.56	0.00
	CL-SCCA with CNN_{TD}	98.31	3.30
	BWCL-SCCA with CNN_{TD}	96.59	1.10

Conclusion

This paper proposed a CL-based SCCA method, called CL-SCCA. As an improvement over DL-SCCA (Staib and Moradi 2023), the proposed method uses unlabeled traces corresponding to multiple byte operations for pretraining, enabling the encoder to capture similar leakage features from the multi-byte leakage distribution. Key differences are then recovered through subsequent fine-tuning and cross-byte transfer. To address the problem that byte-source differences may affect model training during multi-byte pretraining, this paper further proposed a byte-weighted contrastive loss function to improve cross-byte key-difference recovery.

To further evaluate the proposed method, experiments are conducted on five datasets covering three cryptographic primitives, namely AES, SM4, and PRESENT-128, as well as different hardware platforms. Compared with DL-SCCA, CL-SCCA can effectively reduce the key-rank of the correct key difference and improve the success rate of cross-byte key-difference recovery. In addition, CL-SCCA still achieves good attack performance with a simple MLP structure, indicating that contrastive learning can enhance the utilization of multi-byte leakage features in SCCA with relatively low model complexity.

In future work, we will investigate an end-to-end CL-SCCA framework that directly learns collision-related leakage features from raw traces, enabling the model to jointly perform informative leakage-position localization, cross-byte feature alignment, and key-difference recovery.

Abbreviations

SCA	Side-channel analysis
SCCA	Side-channel collision attack
CL	Contrastive learning
DL-SCCA	Deep Learning-based Side-channel Collision Attack
CL-SCCA	Contrastive Learning-based Side-channel Collision Attack
BWCL	Byte-Weighted Contrastive Loss
MLP	Multilayer perceptron
CNN	Convolutional neural network
AES	Advanced Encryption Standard
NT-Xent	Normalized temperature-scaled cross-entropy
RS	Random shift
RC	Random cropping
RD	Random denoising
ReLU	Rectified linear unit
PCA	Principal component analysis
t-SNE	T-distributed stochastic neighbor embedding
BWCL-SCCA	Byte-Weighted Contrastive Learning-based Side-channel Collision Attack

Acknowledgements

Not applicable. The authors confirm that ChatGPT was used only to assist with language polishing and LaTeX formatting. No generative AI tool was used to create scientific ideas, experimental data, experimental analysis, or technical conclusions. All scientific content, analysis, conclusions, and final manuscript decisions were reviewed and approved by the authors.

Author contributions

Zoushaojie Jiang contributed to conceptualization, methodology, experiments, data analysis, and manuscript drafting. An Wang contributed to methodology, validation, and manuscript revision. Annyu Liu contributed to supervision, methodology, and manuscript review. Yaoling Ding (main corresponding author) contributed to supervision, project administration, and manuscript review. Jingqi Zhang contributed to validation and discussion. Jiakun Li contributed to discussion and manuscript revision. Liehuang Zhu contributed to supervision and manuscript review. All authors read and approved the final manuscript.

Funding

This work was supported by the National Cyber Security-National Science and Technology Major Project (No. 2026ZD1500700), National Natural Science Foundation of China (No. 62502035), Beijing Natural Science Foundation (No. L2601014), State Key Laboratory of Cryptography and Digital Economy Security, Shandong University (No. KFZD2503), and BIT Research and Innovation Promoting Project (Grant No. 2025YCXZ041).

Data availability

The datasets supporting the conclusions of this article are available in the ASCAD repository, including ASCAD F and ASCAD R, and in the DPA Contest repository, including DPA Contest v4. The SM4 and PRESENT-128 datasets collected in this work are available from the corresponding author upon reasonable request.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare that they have no competing interests.

Received: 5 June 2026 Accepted: 9 September 2026
Published online: 10 October 2026

References

Agrawal D, Archambeault B, Rao JR, Rohatgi P (2002) The EM side channel (s). In: International workshop on cryptographic hardware and embedded systems. Springer, pp 29–45

ANSSI: ASCAD fixed-key dataset. https://github.com/ANSSI-FR/ASCAD/tree/master/ATMEGA_AES_v1/ATM_AES_v1_fixed_key. Accessed 27 May 2026

ANSSI: ASCAD variable-key dataset. https://github.com/ANSSI-FR/ASCAD/tree/master/ATMEGA_AES_v1/ATM_AES_v1_variable_key. Accessed 27 May 2026

Benadjila R, Prouff E, Strullu R, Cagli E, Dumas C (2020) Deep learning for side-channel analysis and introduction to ASCAD database. J Cryptogr Eng 10(2):163–188

Bogdanov A (2007) Improved side-channel collision attacks on AES. In: International workshop on selected areas in cryptography. Springer, pp 84–95

Cagli E, Dumas C, Prouff E (2017) Convolutional neural networks with data augmentation against jitter-based countermeasures: Profiling attacks without pre-processing. In: International conference on cryptographic hardware and embedded systems. Springer, pp 45–68

Cao P, Zhang C, Lu X, Gu D, Xu S (2022) Improving deep learning based second-order side-channel analysis with bilinear CNN. IEEE Trans Inf Forensics Secur 17:3863–3876

Chen T, Kornblith S, Norouzi M, Hinton G (2020) A simple framework for contrastive learning of visual representations. In: International conference on machine learning. PmLR, pp 1597–1607

Choudary MO, Kuhn MG (2017) Efficient, portable template attacks. IEEE Trans Inf Forensics Secur 13(2):490–501

DPA Contest: DPA Contest v4. https://www.dpacontest.org/v4/. Accessed 27 May 2026

Jiang Z, Wang A, Ding Y, Liu A, Liu Z, Yu J, Zhu L (2025) Extract discriminative features: profiled side-channel analysis for cryptosystems based on supervised contrastive learning. Cryptology ePrint Archive

Jin M, Zheng M, Hu H, Yu N (2020) An enhanced convolutional neural network in side-channel attacks and its visualization. arXiv:2009.08898 arXiv preprint

Kocher PC (1996) Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. Annual International Cryptology Conference. Springer, pp 104–113

Kocher P, Jaffe J, Jun B (1999) Differential power analysis. In: Annual international cryptology conference. Springer, pp 388–397

Kong F, Wang X, Pu K, Zhang J, Dang H (2023) A practical non-profiled deep-learning-based power analysis with hybrid-supervised neural networks. Electronics 12(15):3361

Liu A, Wang A, Sun S, Wei C, Ding Y, Wang Y, Zhu L (2025) Cl-sca: a contrastive learning approach for profiled side-channel analysis. IEEE Trans Inf Forensics Secur

Liu A, Wang A, Liu J, Ding Y, Zhang J, Liu Z, Zhu L (2026) Mask leakage independence: a collision-based profiled approach for side-channel analysis on cryptosystems. IEEE Trans Comput-Aided Des Integr Circ Syst

Llavata D, Cagli E, Eyraud R, Grosso V, Bossuet L (2026) Btcca: unsupervised horizontal attack against RSA primitive with Barlow twins collision correlation analysis. IACR Trans Cryptogr Hardware Embedded Syst 2026(2):574–604

Maghrebi H, Portigliatti T, Prouff E (2016) Breaking cryptographic implementations using deep learning techniques. In: International conference on security, privacy, and applied cryptography engineering. Springer, pp 3–26

Moradi A, Mischke O, Eisenbarth T (2010) Correlation-enhanced power analysis collision attack. In: International workshop on cryptographic hardware and embedded systems. Springer, pp 125–139

Oord Avd, Li Y, Vinyals O (2018) Representation learning with contrastive predictive coding. arXiv preprint arXiv:1807.03748

Ou Y, Li L (2022) Side-channel analysis attacks based on deep learning network. Front Comp Sci 16(2):162303

Schramm K, Wollinger T, Paar C (2003) A new class of collision attacks and its application to DES. In: International workshop on fast software encryption. Springer, pp 206–222

Staib M, Moradi A (2023) Deep learning side-channel collision attack. IACR Trans Cryptogr Hardware Embedded Syst 2023(3):422–444

Timon B (2019) Non-profiled deep learning-based side-channel attacks with sensitivity analysis. In: IACR transactions on cryptographic hardware and embedded systems, pp 107–131

Wu Z, Xiong Y, Yu SX, Lin D (2018) Unsupervised feature learning via non-parametric instance discrimination. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp 3733–3742

Wu L, Tiran S, Perin G, Picek S (2024) Plaintext-based side-channel collision attack

Zaid G, Bossuet L, Dassance F, Habrard A, Venelli A (2021) Ranking loss: maximizing the success rate in deep learning side-channel analysis. In: IACR transactions on cryptographic hardware and embedded systems, pp 25–55

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.